

ExoOS

Kernel • Modules • Serveurs •

Photo

Analyse statique exhaustive — 1059 fichiers Rust, ~363 000 LOC

Ce rapport présente la synthèse master d'un audit de sécurité à froid mené sur l'intégralité du code source d'ExoOS — système d'exploitation Rust `no_std` pour cible `x86_64`. L'audit couvre le bootloader (UEFI + BIOS), le noyau (mémoire, scheduler, IPC, syscall, process, sécurité, crypto), les drivers (réseau, stockage, entrées, affichage, TTY), le système de fichiers (ExoFS, ext4, FAT32, fscrypt, dm-verity), les serveurs userspace (ExoShield NGAV, crypto_server, VFS, IPC router, scheduler server) et le dynamic linker. 9 sous-audits parallèles ont été réalisés, chacun avec lecture exhaustive file-par-file sans survol.

DATE	AUDITEUR	PÉRIMÈTRE
28 juin 2026	Super Z (Z.ai)	9 sous-systèmes + cross-check doc
MÉTHODE	VOLUME	VERDICT
Statique exhaustif à froid	~363 kLOC Rust + 60 docs	4.5 / 10 — non déployable en prod

Table des matières

1. Résumé exécutif	4
1.1 Verdict global	4
1.2 Répartition des vulnérabilités	4
1.3 Verdict par sous-système	4
2. Architecture observée	6
2.1 Couches du système	6
2.2 Défenses en profondeur observées (positif)	6
3. Vulnérabilités critiques consolidées (P0)	8
3.1 Boot chain (B-001 à B-003)	8
3.2 Kernel security (S-001, S-033)	8
3.3 Kernel memory (M-001 à M-004)	8
3.4 ExoShield NGAV (10 critiques)	9
3.5 Crypto (6 critiques)	10
3.6 Filesystem (10 critiques)	10
3.7 Drivers (4 critiques)	11
3.8 Syscall/IPC/Scheduler/Process (5 critiques)	11
4. Patterns transverses (cross-cutting issues)	12
5. Cross-check documentation ↔ code	14
5.1 Correctifs documentés mais non appliqués	14
5.2 Incohérences documentaires	14
6. Plan de remédiation priorisé	15
6.1 P0 — À corriger avant TOUT déploiement	15

6.2 P1 — À corriger avant dev/test avec données sensibles	16
6.3 P2 — À corriger avant release publique	16
6.4 P3 — Amélioration continue	16

7. Conclusion 17

7.1 Verdict final	17
7.4 Score de confiance global : 4.5 / 10	17
7.5 Recommandation finale	17
7.6 Fichiers de référence (findings détaillés)	18

8. Annexes 19

8.1 Méthodologie d'audit	19
8.2 Limites de l'audit	19
8.3 Recommandations d'audit complémentaire	19
8.4 Glossaire	19

1. Résumé exécutif

1.1 Verdict global

L'OS ExoOS dans son état actuel NE DOIT PAS être déployé en production pour un contexte exposé à des attaquants non maîtrisés. Score global : 4.5/10.

L'architecture générale d'ExoOS est saine et plusieurs primitives cryptographiques sont de qualité professionnelle (Ed25519 `verify_strict`, Argon2id conforme OWASP 2024, BLAKE3, ChaCha20 RFC 8439 KAT). Cependant, le code présente un nombre élevé de vulnérabilités critiques non corrigées dans les sous-systèmes essentiels : boot UEFI, mémoire virtuelle, ExoShield, crypto TLS maison, FS POSIX, drivers DMA. Les sections suivantes détaillent les 44 vulnérabilités critiques, 73 majeures et 129 mineures identifiées sur les 9 sous-systèmes audités.

1.2 Répartition des vulnérabilités

Le tableau ci-dessous synthétise la répartition des vulnérabilités par sous-système ainsi que le score de confiance attribué (10 = confiance maximale, 0 = sous-système non fiable). Le score reflète à la fois le nombre de vulnérabilités trouvées et la maturité générale du sous-système — un sous-système avec peu de vulnérabilités mais une architecture faible aura un score moyen.

Sous-système	Fichiers	LOC	Crit	Maj	Min	Score
Boot (exo-boot + loader + kernel boot)	67	~11 200	3	7	8	5.5/10
Kernel security (cap, mitigations, integrity, crypto kernel)	~30	~9 000	2	8	15	5.0/10
Kernel memory (physique, virtuelle, heap, DMA, IOMMU)	~60	~14 000	4	9	16	4.5/10
ExoShield NGAV	47	~12 000	10	13	18	3.0/10
Crypto (kernel + crypto_server + fscrypt + verity)	32	~14 800	6	6	13	4.0/10
Filesystem (ExoFS + ext4 + FAT32 + VFS)	~70	~25 000	10	11	24	3.5/10
Drivers (net, storage, input, display, tty, audio, IOMMU)	71	~15 000	4	11	19	5.0/10
Syscall / IPC / Scheduler / Process + 11 serveurs	~90	~18 000	5	8	16	4.5/10
Cross-check documentation ↔ code	~60 docs	—	—	7 non-appl.	—	6.0/10
TOTAL	~1100	~363 000	44	73	129	4.5/10

1.3 Verdict par sous-système

CRIT

ExoShield (NGAV). Non opérationnel comme défense active. Hooks post-hoc uniquement (observation après exécution), pas de blocage synchrone kernel-side. Un malware déterminé exécute toutes ses actions avant détection.

CRIT	Crypto TLS maison (crypto_server). Cassée. Pas de Finished/transcript hash → MITM trivial. IVs dérivés jamais utilisés. Pas de recv_counter → replay illimité. Module xchacha20_poly1305 ne fait pas de Poly1305 réellement.
CRIT	Filesystem POSIX. Couche permission inexistante. Aucun check uid/gid/mode sur les opérations. FD table globale partagée entre tous les processus. copy_from_user ne valide pas l'origine userspace → contournement SMAP.
CRIT	Mémoire kernel. Race TLB/CoW et validation release désactivée. debug_assert! pour valider user/kernel → en release, mapping arbitraire. Buddy allocator peut perdre des blocs. Swap non chiffré.
CRIT	Boot UEFI. Memory map stale collectée avant allocations kernel → buddy allocator peut réallouer l'image kernel → corruption silencieuse. mark_special_regions() défini mais jamais appelé.
CRIT	Syscall ABI. Escalade Ring3 → Ring1 via spoofing service_class dans sys_exo_ipc_create. UAF dans ProcessRegistry::find_by_pid. sys_uname écrit 390 octets sans valider la borne user.
CRIT	Drivers DMA. IOMMU bypass par défaut dans e1000 et virtio_net via flag BYPASS_IOMMU. e1000 en mode promiscuous par défaut. virtio_net OOB read via head non validé.
CRIT	Kernel security. CET Shadow Stack non fonctionnel (encodage inline asm incorrect — decode en invlpg au lieu de WRSSQ). Escalade capability via delegate() qui vérifie rights.is_subset_of(token.rights()) au lieu des droits stockés en table.
MAJ	Boot BIOS. Stage2 non signé + typo VGA 0xB800_0000 au lieu de 0xB8000.

2. Architecture observée

ExoOS est un système d'exploitation de type micro/noyau hybride en Rust `no_std` pour cible `x86_64`, organisé en 6 couches. Cette section décrit l'architecture observée à partir du code source, puis liste les défenses en profondeur correctement implémentées.

2.1 Couches du système

- 1. exo-boot** — UEFI + BIOS legacy — chargeur de bootstrap signant le kernel via Ed25519+SHA-512 (crate `exo-verity`), avec `verified boot fail-closed` par défaut.
- 2. kernel** — Noyau monolithique modulaire (755 fichiers Rust) couvrant : memory (physique/virtuelle/heap/DMA/IOMMU/COW/swap/NUMA/huge pages), scheduler (CFS-like multi-CPU), IPC (channels, ring buffers, shared memory, RPC, capability bridge), process (fork/exec/exit, namespaces, signals), syscall (handlers + compat POSIX), fs (ExoFS natif + drivers ext4/FAT32), security (capability TIER 0, zero-trust TIER 1, exploit mitigations KPTI/SMEP/SMAP/PKU/UMIP/IBRS/SSBD/retpoline, integrity canary/guard_pages/sanitizer, isolation ExoCage CET, crypto kernel ChaCha20+BLAKE3+HKDF).
- 3. drivers** — Drivers kernel-level (71 fichiers) pour network (e1000, virtio_net, loopback), storage (NVMe, AHCI, virtio_blk, partition GPT), input (PS/2, evdev, USB HID), display (VGA, virtio_gpu), tty (console, pty, vt100, line discipline), audio (HDA, virtio_sound), security (dm-verity), storage crypto (fscrypt).
- 4. servers** — Services userspace (141 fichiers) : `crypto_server` (TLS maison, PKI), `exo_shield` (NGAV), `vfs_server` (translation POSIX ↔ ExoFS), `syscall_abi`, `ipc_router`, `scheduler_server`, `init_server`, `memory_server`, `device_server`, `tty_server`, `fb_server`, `input_server`, `network_server`, `virtio_drivers`, `exosh` (shell).
- 5. loader** — Dynamic linker userspace (20 fichiers) avec sécurité ELF (RELRO, ASLR, PIE, stack canary).
- 6. userspace** — Coreutils + libexo (38 fichiers).

2.2 Défenses en profondeur observées (positif)

Plusieurs aspects méritent d'être soulignés comme correctement implémentés. Ces défenses sont des fondations solides sur lesquelles les correctifs aux vulnérabilités identifiées pourront s'appuyer.

Verified boot Ed25519 strict — `verify_strict` (anti-malléabilité, anti-clé-faible, cofacteur 8) avec `KernelVerdict` enum `fail-closed`. Garde de compilation `key_is_usable()` refuse clés nulles/vecteurs test RFC 8032.

KPTI per-CPU — Kernel Page Table Isolation par CPU avec fallback IRETQ (CVE-2012-0217). Retpoline/IBRS/SSBD Spectre v2/v4 mitigations.

Hardware protections — SMEP/SMAP/PKU/UMIP activées. ExoLedger chaîné BLAKE3 pour audit trail. Layout TCB/SSR avec assertions compile-time exhaustives.

Capability TIER 0 — `RIGHT_ADMIN 0x1FFFFF`, `CapTable::check_object`, fork inherit, mint à l'open, O(1) `verify/revoke`, namespaces.

PRNG ChaCha20 — Conforme RFC 8439 avec test vector officiel. RNG multi-sources (RDRAND + RDSEED + jitter TSC + SP) conditionné BLAKE3.

fscrypt — AEAD XChaCha20-Poly1305 + Argon2id (m=64MiB, t=3, p=4 — dépasse OWASP 2024), wrapping authentifié, comparaison constant-time.

dm-verity Ed25519 — `verify_strict` avec `KernelVerdict fail-closed`, refus des clés de test RFC 8032.

ExoFS superblock — Magic checké en premier (HDR-03), checksum BLAKE3 sur 480 octets, 3 miroirs avec sélection du meilleur par epoch.

Path resolver — Strictement itératif (RECUR-01), borne `max_iters = 8192`, `SYMLINK_MAX_DEPTH = 40`.

TLA+ formelles — 16 specs TLA+ pour preuves de propriétés critiques (ExoOS_Full, ExoShield, Memory, ContextSwitch, SmpBoot, IrqRouting, Adversarial, ExoFS, ExoPhoenixHandoff, CapTokens, ProcessDeath, ExoNet_HardMode).

Code hygiène — `deny(unsafe_op_in_unsafe_fn) + deny(missing_safety_doc)` dans `exo-boot`. Bound checking exhaustif (`checked_add/checked_mul/saturating_sub`) dans `parser ELF boot`.

IOMMU kernel — `fault_queue` lock-free (pattern Vyukov, aligné cache-line 64 octets), `domain_registry` borné 256, SIGKILL automatique sur `fault DMA`.

3. Vulnérabilités critiques consolidées (P0)

Les 44 vulnérabilités critiques ci-dessous permettent une compromission totale du système. Elles doivent être corrigées avant TOUT déploiement.

3.1 Boot chain (B-001 à B-003)

ID	Fichier	Description
B-001	exo-boot/src/bios/vga.rs:23	VGA_BUFFER_BASE = 0xB800_0000 au lieu de 0xB8000 (typo underscore). Boot BIOS silencieux + écritures RAM à ~3 GiB.
B-002	exo-boot/src/main.rs:94→160	Carte mémoire UEFI collectée avant allocations kernel/page-tables, transmise stale au kernel. Les pages du kernel apparaissent comme Usable → buddy allocator peut réallouer l'image kernel → corruption mémoire silencieuse.
B-003	exo-boot/src/memory/regions.rs:209	mark_special_regions() défini mais jamais appelé dans main.rs. Kernel image, page tables et BootInfo ne sont pas marqués réservés.

3.2 Kernel security (S-001, S-033)

ID	Fichier	Description
S-001	kernel/src/security/isolation/exocage.rs:339 + cet.rs:175	CET Shadow Stack entièrement non fonctionnel. L'encodage inline asm .byte 0xF3, 0x48, 0x0F, 0x01, 0x3E est documenté comme WRSSQ [rdi], rsi mais décode en réalité comme invlpg BYTE PTR [rsi]. L'encodage correct est 0x48, 0x0F, 0x38, 0xF6, 0x37. Le kernel croit être protégé par CET mais ne l'est pas. Aucun audit précédent n'avait détecté ce bug car aucun n'avait vérifié l'encodage à l'assembleur.
S-033	kernel/src/security/capability/delegation.rs:57,82,92	Escalade de privilèges capability. delegate() vérifie delegated_rights.is_subset_of(source_token.rights()) au lieu des droits stockés en table. Comme CapToken::from_bytes accepte n'importe quelle valeur de droits (via from_bits_truncate), un attaquant peut forger un token avec rights=ALL, passer le check de subset, et obtenir un nouveau token ALL-droits dans la table cible — escalade READ DELEGATE → ALL.

3.3 Kernel memory (M-001 à M-004)

ID	Fichier	Description
M-001	kernel/src/memory/virtual/address_space/fork_impl.rs	Race condition TLB/CoW : modification des PTEs parent sans shutdown_sync → thread sibling peut écrire sur page COW partagée sans #PF → isolation CoW brisée.
M-002	kernel/src/memory/virtual/address_space/tlb.rs	TlbShutdownQueue::request écrase toujours slot 0 → shutdowns concurrents perdus → stale TLB → use-after-free.
M-003	kernel/src/memory/virtual/address_space/user.rs:137 + kernel.rs:95	debug_assert! pour valider user/kernel → en release, mapping arbitraire kernel ↔ user (contournement SMEP/SMAP/KPTI).
M-004	kernel/src/memory/physical/allocator/buddy.rs:476-500	Blocs perdus quand validation post-pop échoue → fuite + double-allocation possible.

3.4 ExoShield NGAV (10 critiques)

ExoShield est un IDS observationnel, PAS un NGAV opérationnel. Tous les hooks sont post-hoc — ils s'exécutent après l'opération kernel. Aucun mécanisme synchrone n'existe pour bloquer réellement une opération.

ID	Fichier	Description
ES-A-01	hooks/*.rs	Hooks pre*_validate/pre*_check post-hoc : s'exécutent après réception du EVENT_REPORT IPC, donc après que l'opération a eu lieu dans le kernel. Aucun mécanisme synchrone (eBPF/LSM/patchage syscall) n'existe pour bloquer réellement une opération.
ES-B-01	signatures/update_init()	Vide trusted_key_count = 0 → pipeline Ed25519 de mise à jour signatures est mort → seule voie = IPC add_signature non signé (ES-B-02).
ES-B-02	ipc_gate/handlers	IPC add_signature non signé — un malware peut injecter ses propres signatures.
ES-C-01	behavioral/decay_scores	decay_scores existe mais jamais appelé → tous les scores ne font qu'augmenter → à terme ExoShield auto-quarantaine tous les processus (DoS différé).
ES-D-01	ml/mlp_load_trained	Dégrade silencieusement vers des poids LCG aléatoires si checksum mismatch, et boot_log est un no-op (let _ = bytes) → dégradation invisible.
ES-E-01	sandbox/*.rs	Sandbox non-isolante (pas de PID/mount/net namespaces kernel-side).
ES-F-01	network/*.rs	Pas d'inspection TLS — MITM impossible à détecter.
ES-G-01	engine/evaluate_policy	Retourne Allow inconditionnellement pour src_pid == 0 (kernel) — bypass total si spoofing possible.
ES-H-01	engine/compute_threat_score	sig_c / 1000 * WEIGHT (division entière tronquée à 0) → scoring binaire, granularité 0..1000 perdue.
ES-J-01	(architecture)	Pas de watchdog kernel pour redémarrer ExoShield — un malware peut le tuer.
ES-K-01	(architecture)	TOCTOU entre scan et exec — un malware peut remplacer le fichier entre le scan et l'exec.

3.5 Crypto (6 critiques)

La stack TLS maison du `crypto_server` est cassée à un niveau fondamental. MITM trivial, replay illimité, module Poly1305 menteur. Recommandation : remplacer par rustls ou ring.

ID	Fichier	Description
C-01	<code>crypto_server/src/tls/</code>	TLS 1.3 maison sans Finished/transcript hash → MITM trivial.
C-02	<code>crypto_server/src/tls/</code>	IVs TLS dérivés jamais utilisés ; nonce = compteur global partagé.
C-03	<code>crypto_server/src/tls/</code>	Aucun <code>recv_counter</code> → replay illimité.
C-04	<code>crypto_server/src/xchacha20_poly1305.rs</code>	Module ne fait PAS de Poly1305 — utilise BLAKE3 keyed-hash (nom menteur, pas d'interop).
C-05	<code>crypto_server/src/tls/</code>	Clé privée X25519 éphémère stockée dans <code>session.server_random</code> (champ exposé).
C-06	<code>kernel/src/security/crypto/aes_ni.rs</code>	AES-NI inline asm ne save xmm que 0/1, pas de cli/sti → corruption FPU si préemption.

3.6 Filesystem (10 critiques)

ID	Fichier	Description
F-001	<code>kernel/src/fs/exofs/syscall/validation.rs:194-220</code>	<code>copy_from_user/copy_to_user</code> ne valident pas que l'adresse est userspace → contournement SMAP, kernel R/W arbitraire.
F-002	<code>syscall/object_{open,read,write,delete,set_meta}.rs</code>	Aucune vérification des permissions POSIX (mode/uid/gid) sur l'ensemble des opérations FS.
F-003	<code>syscall/object_open.rs:36-42, object_fd.rs:30-33</code>	<code>OpenArgs.owner_uid</code> fourni par l'appelant userspace → élévation de privilèges triviale.
F-004	<code>syscall/object_delete.rs:170-173, object_set_meta.rs:364-366</code>	<code>delete/set_meta</code> par chemin bypassent les capacités (TIER 1 non implémenté).
F-005	<code>syscall/object_fd.rs:39-52</code>	FD table globale partagée entre tous les processus (pas de fd namespace par-process).
F-006	<code>syscall/import_object.rs:69-88</code>	<code>read_user_buf</code> lit octet-par-octet sans <code>access_ok</code> , panic kernel sur page unmapped.
F-007	<code>drivers/fs/src/ext4/superblock.rs:152</code>	$1024u32 \ll s_log_block_size$ — UB Rust si $s_log_block_size \geq 32$ (FS malveillant).
F-008	<code>drivers/fs/src/ext4/extent.rs:48-78</code>	Boucle <code>0..eh_entries</code> non-bornée → OOB read kernel + <code>&*</code> sur <code>repr(C, packed)</code> = UB.
F-009	<code>drivers/fs/src/ext4/dir.rs:29</code>	<code>&*(packed)</code> sur <code>Ext4DirEntry2</code> — UB Rust sur struct packed.
F-010	<code>compress/{lz4,zstd}_wrapper.rs, decompress_reader.rs:131</code>	Decompression bombs : <code>original_size</code> non-borné, allocation jusqu'à 4 GiB.
F-020	(architecture)	dm-verity absent — seul l'image kernel est signée, pas le root FS.

3.7 Drivers (4 critiques)

ID	Fichier	Description
D-CRIT-001	drivers/network/e1000 + virtio_net + memory/dma/core/mapping.rs:195	DMA IOMMU bypass par défaut. e1000 et virtio_net utilisent le flag BYPASS_IOMMU (bit 4) pour leurs anneaux DMA. Le kernel honore ce flag en retournant IovaAddr = phys_addr direct — pas de translation IOMMU. Un device compromis (ou PCIe Thunderbolt malveillant) peut DMA vers n'importe quelle adresse physique → compromission totale.
D-CRIT-002	drivers/network/e1000/src/main.rs	program_rx active RCTL_UPE RCTL_MPE RCTL_SBP RCTL_BAM — unicast + multicast promiscuous + store-bad-packets. Le NIC accepte toutes les trames y compris bad-CRC. DoS, spoofing, privacy.
D-CRIT-010	drivers/network/virtio_net/src/main.rs	poll_rx et poll_tx indexent rx_pool_for_head[head as usize] où head vient de elem.id as u16 lu depuis la used ring DMA (device-controlled). Pas de validation head < queue_size. Un device virtio malveillant peut déclencher panic kernel ou corruption logique.
D-CRIT-016	drivers/storage/virtio_blk/src/legacy_pci.rs	Wrappers inb/inw/inl/outb/outw/outl utilisent options(nomem, nostack, preserves_flags). nomem permet au compilateur de réordonner des accès mémoire autour des opérations I/O port → notification device avant écriture buffer DMA = corruption silencieuse.

3.8 Syscall/IPC/Scheduler/Process (5 critiques)

ID	Fichier	Description
Y-CRIT-01	kernel/src/syscall/handlers/exo_ipc.rs (sys_exo_ipc_create)	Escalade Ring3 → Ring1 via spoofing service_class. N'importe quel process peut s'enregistrer comme memory_server et obtenir can_inject_src_pid=true permettant de forger sender_pid=1 dans les IPC.
Y-CRIT-02	kernel/src/process/core/registry.rs (ProcessRegistry::find_by_pid)	Use-after-free : aucun incrément de refcount avant retour de référence &ProcessControlBlock; ; race kill + wait4 exploitable.
Y-CRIT-03	kernel/src/syscall/handlers/uname.rs	sys_uname écrit 390 octets via core::ptr::write_bytes sans valider buf_ptr + 390 ≤ USER_ADDR_MAX (dépassement dans espace kernel).
Y-CRIT-04	kernel/src/syscall/handlers/arch_prctl.rs	sys_arch_prctl(ARCH_SET_FS/SET_GS) sans validation de canonicité ni de privilege.
Y-CRIT-05	kernel/src/syscall/handlers/sigreturn.rs (verify_and_extract_uc)	Déréféréncie un pointeur user sans copy_from_user (violation SMAP) + OOB read possible vers kernel memory.

4. Patterns transverses (cross-cutting issues)

Ces patterns affectent plusieurs sous-systèmes et révèlent des défauts structurels. Les corriger nécessite des changements architecturaux transversaux plutôt que des patches ponctuels.

4.1 Validation release-vs-debug absente

Plusieurs assertions critiques (séparation user/kernel, bornes de buffers) utilisent `debug_assert!` qui est désactivé en release builds. Exemples : M-003 (`debug_assert!` pour valider user/kernel addresses), `patch_04` (drop silencieux au lieu de panic en release), M-004 (validation post-pop buddy désactivée en release).

Recommandation : Toutes les assertions de sécurité doivent utiliser `assert!` (ou `const _: () = assert!(...)` pour les checks compile-time).

4.2 Hooks/IPC post-hoc sans enforcement synchrone

ExoShield (ES-A-01) : hooks post-hoc uniquement. Zero-trust TIER 1 : bypass pour 3 fast-path syscalls. Capability TIER 1 FS : `delete/set_meta` par chemin bypassent les caps (F-004).

Recommandation : Implémenter un mécanisme synchrone kernel-side (eBPF/LSM équivalent ExoOS) avec hook `pre_*` bloquant avant exécution.

4.3 Confiance aveugle en données device-controlled

D-CRIT-010 : `virtio_net` truste head from used ring DMA. D-MAJ-008 : `virtio_net` truste `addr/len` des descripteurs. D-CRIT-001 : IOMMU bypass par défaut. F-008 : `ext4` truste `eh_entries` sans borne.

Recommandation : Tout champ lu depuis device-controlled memory ou on-disk structure doit être borné avant usage (`if head < queue_size { ... } else { return Err; }`).

4.4 Crypto maison non standard

C-01 à C-06 : TLS 1.3 maison sans Finished/transcript hash. C-04 : Module `xchacha20_poly1305` ne fait pas Poly1305. C-09 : Certificats basés sur TSC (non fiable).

Recommandation : Utiliser exclusivement des crates RustCrypto auditées (`aes-gcm`, `chacha20poly1305`, `rustls`) ou ring. Ne jamais implémenter TLS from scratch.

4.5 Encodage inline asm non vérifié

S-001 : CET WRSSQ encodage incorrect (décode en `invlpg`). D-CRIT-016 : nomem sur wrappers I/O port permet réordonnancement compilateur.

Recommandation : Tout inline asm doit être testé via désassemblage automatique en CI (`objdump -d + pattern match`).

4.6 État global sans namespace par-process

F-005 : FD table globale partagée. Y-CRIT-01 : `service_class` spoofable. Y-CRIT-02 : `ProcessRegistry` sans refcount atomique.

Recommandation : Tout état global doit être partitionné par PID ou par capability, avec refcount atomique.

4.7 Entropie et zeroization insuffisantes

M-012 : Stack canary basé sur RDTSC uniquement. C-07 : VOLUME_KEY jamais zéroisée. C-10 : Drop for KeyEntry non-volatile. Reseed post-Phoenix non câblé (CORR-12).

Recommandation : Utiliser zeroize crate avec ZeroizeOnDrop, mélanger RDRAND+RDSEED+jitter, reseed post-Phoenix obligatoire.

5. Cross-check documentation ↔ code

Sur 58 correctifs documentés comme appliqués dans les audits précédents (Claude alpha/beta/gamma/iota/delta, ChatGPT, Kimi-AI), **51 sont effectivement appliqués (88%)**, 4 partiellement appliqués, 3 non appliqués, et 1 introuvable. Cette section liste les correctifs documentés mais manquants dans le code, qui représentent un risque particulier car les audits précédents les considéraient comme résolus.

5.1 Correctifs documentés mais non appliqués

Fix ID	Description	Statut
patch_06 ExoNmi watchdog	Timeout watchdog hardcoded, ni watchdog_set_timeout, ni WATCHDOG_TIMEOUT_MIN_NS	NON-APPLIQUÉ
CORR-54 CRITICAL_SERVERS	Liste absente de servers/, politique de gel Phoenix non contractualisée	NON-APPLIQUÉ
Reseed post-Phoenix (CORR-12/GI-05)	PhoenixWakeEntropy jamais câblé — risque de réutilisation de nonces après restore	NON-APPLIQUÉ
CVE-EX0-001 (patch_01)	Spin-wait SECURITY_READY présent mais ap_wait_security_ready() avec timeout TSC + freeze_req(SecurityInitTimeout) absent	PARTIEL
patch_02 CET per-thread	enable_cet_for_thread existe mais n'est jamais appelé depuis ThreadControlBlock::new()	PARTIEL
patch_04 ExoLedger overflow	Drop silencieux au lieu de freeze_req(LedgerFull), docstring obsolète mentionnant « panique »	PARTIEL
CORR-10 IPI excl Kernel B	KERNEL_B_APIC_ID constante jamais définie en code, exclusion explicite Core 0 absente	PARTIEL

5.2 Incohérences documentaires

Au-delà des correctifs manquants, plusieurs incohérences entre la documentation et le code ont été identifiées. Ces incohérences peuvent induire en erreur les développeurs et les futurs auditeurs, et doivent être résolues pour rétablir la confiance dans la documentation.

- ExoPhoenix_Spec_v6.md obsolète (MAX_CORES=64 vs lib 256).
- MAX_CORES_RUNTIME=64 coexiste avec MAX_CORES_LAYOUT=256 (intentionnel mais ambigu).
- libs/exo-types/ et libs/exo-ipc/ ciblés par plusieurs patches mais absents de l'arbre audité.
- kstack_top stocké dans _cold_reserve[32..40] (offset 176) au lieu de l'offset 144 documenté (ALPHA-03).

6. Plan de remédiation priorisé

6.1 P0 — À corriger avant TOUT déploiement

Ces 31 correctifs doivent être appliqués avant toute considération de déploiement, même en dev/test. Estimation : 3-6 semaines-homme pour un développeur expérimenté en OS Rust.

ID	Action corrective
B-001	VGA typo (1 ligne à corriger) : 0xB800_0000 → 0xB8000
B-002/B-003	Re-collecter memory map UEFI après allocations OU appeler mark_special_regions()
S-001	Remplacer les 5 bytes WRSSQ dans cet.rs:175 et exocage.rs:339 par 0x48, 0x0F, 0x38, 0xF6, 0x37
S-033	Corriger delegation.rs:57,82,92 pour utiliser table.get(oid).rights au lieu de token.rights()
M-001/M-002	Ajouter shutdown_sync dans fork_impl.rs + fixer TlbShutdownQueue::request (multi-slot)
M-003	Convertir debug_assert! user/kernel en assert!
M-004	Fixer buddy allocator validation post-pop
ES-A-01	Implémenter hooks kernel synchrones (eBPF/LSM equivalent ExoOS)
ES-B-01/B-02	Pré-charger trusted keys Ed25519 au boot + exiger signature pour add_signature IPC
ES-C-01	Appeler decay_scores depuis perform_maintenance
ES-D-01	Logger l'échec de mlp_load_trained au lieu de dégrader silencieusement
ES-E-01	Implémenter vraie sandbox (PID/mount/net namespaces kernel-side)
ES-G-01	Refuser src_pid == 0 dans evaluate_policy sauf vérification explicite
ES-H-01	Corriger compute_threat_score : (sig_c * WEIGHT) / 1000
ES-J-01	Implémenter watchdog kernel pour redémarrer ExoShield
C-01 à C-06	Réécrire TLS avec rustls ou implémenter Finished/transcript hash + recv_counter + vrai Poly1305
F-001	Ajouter access_ok dans copy_from_user/copy_to_user
F-002/F-003/F-004	Implémenter checks POSIX uid/gid/mode + capability TIER 1 sur delete/set_meta par chemin
F-005	FD table par-process
F-007/F-008/F-009	Validation stricte structures on-disk ext4 (borne s_log_block_size < 32, eh_entries borné, suppression &* sur packed)
F-010	Borne original_size (max 256 MiB) sur decompression
F-020	Implémenter dm-verity pour root FS
D-CRIT-001	Supprimer BYPASS_IOMMU des drivers e1000/virtio_net + faire ignorer ce flag par le kernel pour PID != 0
D-CRIT-002	Désactiver RCTL_UPE RCTL_MPE RCTL_SBP par défaut dans e1000
D-CRIT-010	Valider head < queue_size dans virtio_net poll_rx/poll_tx
D-CRIT-016	Retirer nomem des wrappers asm! I/O port dans legacy_pci.rs
Y-CRIT-01	Authentifier service_class par capability token dédié
Y-CRIT-02	Incrémenter refcount avant retour de find_by_pid

ID	Action corrective
Y-CRIT-03	Valider <code>buf_ptr + 390 ≤ USER_ADDR_MAX</code> dans <code>sys_uname</code>
Y-CRIT-04	Valider canonicité + privilege dans <code>arch_prctl</code>
Y-CRIT-05	Utiliser <code>copy_from_user</code> dans <code>verify_and_extract_uc</code>

6.2 P1 — À corriger avant dev/test avec données sensibles

Les 73 vulnérabilités majeures détaillées dans les 9 rapports individuels (B-004 à B-010, S-002 à S-032, M-005 à M-012, ES-A-05, ES-C-01, ES-D-02, ES-F-01, ES-G-01, ES-H-02, C-07 à C-12, F-011 à F-021, D-MAJ-001 à D-MAJ-028, Y-HIGH-01 à Y-HIGH-08). Estimation : 2-3 mois supplémentaires.

6.3 P2 — À corriger avant release publique

Les 65 vulnérabilités moyennes + correctifs documentaires (CORR-54, CORR-12, patch_06, patch_02, patch_04, CORR-10) + incohérences doc. Estimation : 1-2 mois supplémentaires.

6.4 P3 — Amélioration continue

Les 64 vulnérabilités mineures + 11 INFO + mise à jour docs obsolètes. Estimation : en continu après release.

7. Conclusion

7.1 Verdict final

ExoOS présente une architecture de sécurité ambitieuse et bien pensée sur le papier, mais l'implémentation actuelle comporte de nombreux défauts critiques qui la rendent impropre à un déploiement production.

7.2 Forces indéniables du projet

- Choix techniques sains (Rust no_std, capability-based, zero-trust, micro/noyau hybride).
- Primitives cryptographiques de qualité professionnelle (Ed25519 strict, Argon2id conforme OWASP, BLAKE3, ChaCha20 KAT).
- Défenses en profondeur multiples (KPTI, SMEP/SMAP, PKS, retpoline, IBRS, SSBD, guard pages, canary, ExoLedger).
- Spécifications TLA+ formelles pour preuves de propriétés.
- Verified boot avec fail-closed par construction et garde anti-régression compile-time.

7.3 Faiblesses structurelles

- ExoShield (NGAV) n'est pas opérationnel : hooks post-hoc uniquement, pas de blocage synchrone.
- TLS maison cassée : MITM trivial, replay illimité, module Poly1305 menteur.
- Couche POSIX FS inexistante : pas de checks uid/gid/mode, FD table globale.
- CET Shadow Stack non fonctionnel : encodage inline asm incorrect.
- IOMMU bypass par défaut dans drivers DMA.
- Validation user/kernel en debug_assert! désactivée en release.
- Plusieurs correctifs documentés comme appliqués ne le sont pas réellement (CORR-54, CORR-12, patch_06, patch_02 partiel, patch_04 partiel, CORR-10 partiel).

7.4 Score de confiance global : 4.5 / 10

Pour usage recherche/développement isolé (pas d'attaquant, pas de données sensibles) : **acceptable** avec prudence.

Pour usage dev/test avec données sensibles : **non recommandé** tant que P0 + P1 ne sont pas corrigés.

Pour usage production exposée : **formellement proscrit** tant que P0 n'est pas traité.

7.5 Recommandation finale

Appliquer les 31 correctifs P0 listés en section 6 avant toute considération de déploiement. Compter environ **3-6 semaines-homme** pour un développeur expérimenté en OS Rust pour traiter P0 seul, et **3-6 mois** pour P0+P1+P2.

L'audit précédent (Claude/Kimi/ChatGPT) a identifié la majorité des patterns mais a manqué plusieurs bugs critiques que cet audit à froid a permis de découvrir : S-001 (CET WRSSQ encodage) — invisible sans désassemblage ; C-01 à C-06 (TLS maison cassée) — invisible sans lecture du module TLS line-by-line ; F-001 (copy_from_user sans access_ok) — invisible sans lecture du validation.rs ; D-CRIT-016 (nomem sur asm I/O port) — invisible sans connaissance fine de asm! options ; ES-A-01 (hooks post-hoc) — invisible sans comprendre la sémantique du flux IPC EVENT_REPORT ; M-003 (debug_assert pour user/kernel) — invisible sans différence release-vs-debug.

7.6 Fichiers de référence (findings détaillés)

Ce rapport de synthèse s'appuie sur 9 rapports d'audit détaillés totalisant environ 6800 lignes. Chaque rapport est disponible dans le dossier `/home/z/my-project/audit_mem/` et fournit l'inventaire exhaustif des fichiers audités, les descriptions détaillées des vulnérabilités avec numéros de ligne, les preuves de concept théoriques, et les recommandations spécifiques.

Rapport	Chemin	Lignes
Cross-check documentation ↔ code	audit_mem/02_DOC_crosscheck.md	324
Boot chain	audit_mem/03_BOOT_findings.md	570
Kernel security	audit_mem/04_SEC_findings.md	606
Kernel memory	audit_mem/04_MEM_findings.md	981
ExoShield NGAV	audit_mem/05_SHIELD_findings.md	662
Crypto	audit_mem/06_CRYPT0_findings.md	853
Filesystem	audit_mem/07_FS_findings.md	852
Drivers	audit_mem/08_DRV_findings.md	878
Syscall/IPC/Scheduler/Process	audit_mem/09_SYS_findings.md	1009
Synthèse master (ce document)	audit_mem/00_MASTER_AUDIT_REPORT.md	~800

8. Annexes

8.1 Méthodologie d'audit

L'audit a été conduit selon une méthodologie en 5 étapes : (1) extraction de l'archive source exoos.zip (1059 fichiers Rust, ~363 kLOC) ; (2) cartographie des sous-systèmes (kernel, servers, drivers, loader, exo-boot, userspace) ; (3) 9 audits parallèles spécialisés par sous-système, chacun avec lecture exhaustive file-par-file sans survol ; (4) 1 cross-check documentation ↔ code ; (5) synthèse master consolidant tous les findings.

8.2 Limites de l'audit

Cet audit est statique, à froid, lecture-seule. Aucune modification du code source n'a été effectuée. Les limitations sont :

- Pas de tests dynamiques (fuzzing, exploitation réelle).
- Pas de revue des dépendances Cargo externes (RustCrypto, etc.) — supposées de confiance.
- Pas de revue des specs matérielles (Intel SDM, AMD APM) — supposées correctes.
- Pas d'analyse de timing micro-architectural.
- Tests TLA+ présents mais non rejoués.

8.3 Recommandations d'audit complémentaire

- **Fuzzing** : cargo fuzz sur parsers (ELF, ext4, FAT32, GPT, BPB, TLS, X.509, Ed25519 signatures).
- **Tests dynamiques** : exploitation réelle des P0 sur QEMU.
- **Revue dépendances** : cargo audit + cargo deny.
- **Vérification TLA+** : rejouer les 16 specs avec TLC sur des modèles plus larges.
- **Analyse side-channel** : instrumenter les chemins crypto avec dudedct.
- **Conformance boot** : tester le secure boot avec UEFI Spec Compliance Suite.

8.4 Glossaire

Terme	Définition
CET	Control-flow Enforcement Technology (Intel) — shadow stack + indirect branch tracking.
KPTI	Kernel Page Table Isolation (Meltdown mitigation).
NGAV	Next-Generation Antivirus.
PKS	Protection Keys for Supervisor (PKU kernel variant).
SMAP/SMEP	Supervisor Mode Access/Execution Prevention.
TLB	Translation Lookaside Buffer.
TOCTOU	Time-Of-Check vs Time-Of-Use.
UMIP	User-Mode Instruction Prevention.
UAF	Use-After-Free.